

Struktur Data dan Algoritma — Laporan Proyek Kelompok

Implementasi Graph untuk Penyelesaian Masalah Navigasi dan Optimasi Waktu Bus UNS

Grecia Aulia Hermawan (L0225003), Stiven Arya Duta Wicaksono (L0225011), Naila Zhafira Rahma Widyatmoko (L0225027)

Informasi Project

Kelas: A

Prodi: S1 Sains Data

Fakultas: Teknologi Informasi dan Sains Data

Tautan repo/artefak: <https://github.com/stivenarya27-arch/BUS-UNS-JAVA>

Abstract

Layanan bus kampus Universitas Sebelas Maret (UNS) berperan penting dalam mendukung mobilitas civitas akademika. Namun, keterbatasan informasi mengenai posisi bus, kapasitas armada serta estimasi waktu kedatangan menyebabkan perjalanan kurang efisien. Project ini mengembangkan sistem informasi bus kampus berbasis struktur data dan algoritma untuk membantu pengguna memperoleh informasi perjalanan yang lebih akurat. Struktur data yang digunakan meliputi Graph untuk merepresentasikan jaringan halte dan rute bus, ArrayList sebagai adjacency list serta Disjoint Set (Union - Find) untuk mengelompokkan halte berdasarkan jalur operasional. Estimasi waktu tempuh dihitung menggunakan algoritma Depth-First Search (DFS). Hasil pengujian menunjukkan bahwa sistem mampu menghitung waktu perjalanan, memvalidasi rute, mendeteksi perjalanan lintas jalur dan menangani masukan (inputan) tidak valid dengan benar. Dengan demikian sistem ini mendukung layanan transportasi kampus yang lebih efisien dan informatif.

Kata kunci: Graph, ArrayList, Depth-First Search, Disjoint Set, Sistem Informasi Bus Kampus

1 Pendahuluan & Rumusan Masalah

- **Definisi Masalah:**

Layanan bus kampus Universitas Sebelas Maret (UNS) pada kondisi awalnya hanya mengoperasikan satu unit bus dengan rute sirkular yang melingkupi seluruh area kampus. Hal ini mengakibatkan penumpukan penumpang, waktu tunggu yang lama, dan ketidakefisienan operasional secara keseluruhan. Selain itu, belum tersedia pula sistem informasi yang memadai bagi penumpang, sehingga penumpang tidak dapat mengetahui

posisi bus secara real-time, kondisi kapasitas armada yang sedang beroperasi maupun estimasi waktu kedatangan bus di masing-masing halte untuk merencanakan perjalanan secara efisien.

Untuk mengatasi permasalahan tersebut, kampus UNS mengoperasikan dua unit bus yang masing-masing melayani jalur barat dan jalur timur secara paralel. Guna mendukung operasional tersebut, dikembangkan sistem informasi berbasis struktur data dan algoritma yang mampu menampilkan posisi bus secara real-time, menginformasikan kapasitas armada, serta menghitung estimasi waktu kedatangan dan durasi perjalanan secara dinamis. Dengan adanya sistem ini pengguna dapat merencanakan perjalanan dengan lebih efisien.

- **Motivasi:**

Universitas Sebelas Maret (UNS) memiliki area kampus yang luas dengan berbagai fasilitas yang tersebar, sehingga keberadaan bus kampus menjadi kebutuhan nyata. Namun, pengelolaan rute bus kampus yang tidak terstruktur kerap menimbulkan permasalahan, di antaranya ketidakpastian kedatangan bus di setiap halte, ketidaktahuan penumpang terhadap kondisi kapasitas bus yang sedang beroperasi, serta sulitnya merencanakan perjalanan secara efisien tanpa informasi yang memadai.

Permasalahan tersebut dapat diselesaikan dengan memanfaatkan konsep struktur data dan algoritma. Konsep tersebut sangat relevan karena mampu melakukan penghitungan matematis, pemrosesan data secara terstruktur, dan penyajian informasi berbasis data. Tanpa penerapan struktur data dan algoritma, informasi seperti estimasi waktu kedatangan dan status kapasitas bus hanya dapat diperoleh melalui perkiraan manual yang tidak akurat. Sebaliknya, dengan penerapan struktur data dan algoritma, seluruh perhitungan mengikuti kondisi terkini, sehingga pengguna dapat mengambil keputusan berbasis data.

- **Batasan:**

1. Untuk jalur timur, bus akan mengakomodasi halte : UNS Inn, FT, FEB, FH, dan Pascasarjana, sedangkan untuk jalur barat, bus akan mengakomodasi halte : Gerbang Utara, FK, FMIPA, dan FP.
2. Jalur yang ditentukan tetap.
3. Arah pergerakan bus searah.
4. Untuk jalur timur bergerak dari arah gerbang belakang menuju gerbang depan, sedangkan untuk jalur barat bergerak dari arah gerbang depan menuju gerbang belakang.
5. Waktu yang tersedia merupakan estimasi.

6. Jumlah penumpang tidak diketahui (hanya diketahui penuh atau tidak)
7. Bus kampus tidak memungut biaya.
8. Jika bus penuh, asumsikan pada halte berikutnya terdapat kursi yang kosong.
9. Jika bus penuh, asumsikan tidak ada orang yang akan turun di halte tempat penumpang akan naik.
10. Waktu real (waktu yang tercatat [WIB]) tidak ditampilkan, yang ditampilkan hanya estimasi kedatangan bus dan waktu yang diperlukan untuk menempuh perjalanan.
11. Sistem hanya memiliki dua peran pengguna, yaitu supir bus dan penumpang.
12. Tidak mendukung perjalanan lintas jalur. Penumpang yang berada di jalur barat tidak dapat merencanakan perjalanan ke halte yang berada di jalur timur, begitu juga sebaliknya.

2 Desain Solusi & Struktur Data

Sistem ini dibangun menggunakan kombinasi beberapa struktur data yang dipilih berdasarkan kebutuhan representasi rute, penyimpanan hubungan antar halte dan validasi jalur perjalanan .

- **Pemilihan Struktur Data:**

- | | |
|------------------------------|---|
| Graph | : Graph merupakan struktur data yang terdiri atas sekumpulan simpul (vertex) dan hubungan antar simpul atau sisi (edge). Graph digunakan untuk merepresentasikan hubungan antar objek. |
| ArrayList | : ArrayList adalah struktur data yang digunakan untuk menyimpan kumpulan data berbentuk array yang ukurannya dapat berubah sesuai jumlah data yang dimasukkan. |
| Union-Find
(Disjoint Set) | : Disjoint Set merupakan struktur data yang digunakan untuk mengelompokkan elemen-elemen ke dalam beberapa himpunan yang saling lepas (tidak beririsan), serta menentukan apakah dua elemen berada dalam himpunan yang sama melalui operasi find dan union. |

- **Justifikasi:**

Graph : Graph dipilih karena permasalahan ini pada dasarnya adalah jaringan halte yang saling terhubung, di mana setiap halte berperan sebagai simpul (vertex) dan jalur antar halte berperan sebagai sisi (edge) berbobot yang merepresentasikan waktu tempuh, sehingga penelusuran jalur dan perhitungan estimasi waktu dapat dilakukan secara sistematis. Selain itu, terdapat penerapan graph berarah pada kasus ini yang menunjukkan arah pergerakan bus dari satu halte menuju halte lain.

ArrayList : ArrayList digunakan untuk merepresentasikan graph dalam bentuk adjacency list, di mana setiap halte menyimpan daftar Edge berupa halte tujuan dan bobot waktu tempuhnya. Representasi ini dipilih karena jumlah koneksi antar halte relatif sedikit dan tetap dibandingkan total kemungkinan koneksi yang ada.

Union-Find (Disjoint Set) : Disjoint Set dipilih karena sistem ini membagi seluruh halte ke dalam dua kelompok jalur yang terpisah, yaitu jalur barat dan jalur timur, sehingga diperlukan cara untuk mengecek dengan cepat apakah suatu halte termasuk ke dalam jalur yang sama dengan halte lainnya. Struktur ini memungkinkan validasi tersebut dilakukan secara efisien, sehingga sistem dapat menolak perjalanan lintas jalur secara otomatis.

3 Desain Algoritma & Pseudocode

Algoritma utama yang digunakan dalam sistem ini adalah Depth-First Search (DFS).

Menurut Weiss DFS merupakan generalisasi dari preorder traversal pada pohon yang diperluas untuk diterapkan pada struktur graf. Pada DFS, penelusuran dimulai dari vertex v , kemudian masuk sedalam mungkin ke dalam graf, melainkan mengikuti satu jalur ke dalam hingga tidak ada lagi vertex yang bisa dikunjungi. Setelah selesai, DFS melakukan backtrack ke vertex sebelumnya dan mencoba jalur lain. Karena graf dapat memiliki siklus, diperlukan penanda visited agar setiap vertex hanya dikunjungi satu kali. Proses ini terus berulang hingga seluruh vertex yang dapat dijangkau telah dikunjungi.

Pada program Sistem Bus Kampus UNS ini, algoritma DFS diterapkan melalui fungsi `dfs()` yang dipanggil oleh fungsi `hitungWaktu()`. DFS digunakan untuk menelusuri graf halte bus guna menghitung estimasi waktu tempuh antar halte. Penelusuran dimulai dari halte posisi bus saat ini sebagai vertex awal, kemudian masuk sedalam mungkin mengikuti satu jalur rute bus hingga menemukan halte tujuan. Setiap kali `hitungWaktu()` dipanggil, array penanda `new boolean[TOTAL]` dibuat secara langsung sebagai argumen untuk menandai halte mana saja yang sudah dilalui selama penelusuran berlangsung. Hal ini diperlukan karena rute bus berbentuk sirkular sehingga berpotensi menimbulkan siklus yang dapat menyebabkan penelusuran berputar tanpa henti. Selama penelusuran, waktu tempuh terus diakumulasikan setiap kali berpindah dari satu halte ke halte berikutnya sesuai bobot waktu pada setiap edge. Apabila halte tujuan ditemukan, hasil waktu tempuh disimpan ke dalam `h[0]` dan penelusuran langsung dihentikan.

Algoritma

FUNCTION `hitungWaktu(dari, ke)`:

```
    IF dari == ke THEN
        RETURN 0
    END IF
    hasil[0] ← -1
    visited[] ← false
    CALL dfs(dari, ke, 0, visited, hasil)
    RETURN hasil[0]
END FUNCTION
```

FUNCTION `dfs(current, tujuan, waktu, visited[], hasil[])`:

```
    IF current == tujuan THEN
        hasil[0] ← waktu
        RETURN
    END IF
    visited[current] ← true
    FOR EACH edge IN graph[current] DO
        IF visited[edge.tujuan] == false THEN
            CALL dfs(edge.tujuan, tujuan, waktu + edge.waktu, visited, hasil)
            IF hasil[0] != -1 THEN
                RETURN
            END IF
        END IF
    END FOR
END FUNCTION
```

Fungsi hitungWaktu(dari, ke)

Pertama, periksa apakah halte asal sama dengan halte tujuan. Jika sama, langsung kembalikan nilai 0 karena tidak perlu perjalanan. Jika berbeda, siapkan variabel hasil dengan nilai awal -1 sebagai penanda belum ditemukan, dan siapkan array penanda kunjungan yang seluruhnya bernilai false. Kemudian panggil fungsi DFS dengan halte asal sebagai titik mulai, lalu kembalikan hasil yang tersimpan.

Fungsi dfs(current, tujuan, waktu, visited[], hasil[])

Pertama, periksa apakah halte yang sedang dikunjungi saat ini adalah halte tujuan. Jika iya, simpan total waktu yang telah ditotal ke dalam hasil, lalu hentikan penelusuran. Jika belum, tandai halte saat ini sebagai sudah dikunjungi agar tidak dikunjungi lagi. Selanjutnya, periksa satu per satu semua halte tetangga yang terhubung dari halte saat ini. Untuk setiap tetangga yang belum dikunjungi, lakukan penelusuran lebih dalam secara rekursif dengan menambahkan waktu tempuh edge yang dilalui ke total waktu. Apabila setelah penelusuran rekursif tersebut hasil sudah ditemukan (bukan -1), hentikan penelusuran dan kembali ke pemanggil. Jika belum ditemukan, lanjutkan mencoba tetangga berikutnya hingga semua kemungkinan jalur habis ditelusuri.

4 Implementasi Program

- **Lingkungan:**

Program ini dikembangkan menggunakan bahasa pemrograman Java dan dieksekusi melalui lingkungan Java Development Kit (JDK). Pemilihan Java didasarkan pada dukungannya terhadap paradigma pemrograman berorientasi objek, sehingga struktur data seperti Graph, ArrayList, dan Disjoint Set dapat diimplementasikan secara eksplisit dalam bentuk kelas-kelas yang saling berelasi, serta kemampuan rekursi yang dibutuhkan dalam penerapan algoritma DFS. Program ini dijalankan melalui antarmuka berbasis teks pada command line. Di mana seluruh interaksi antara pengguna dan program berlangsung melalui input dan output standar.

- **Struktur Kode:**

Program dibanmngun dalam satu berkas dengan oembagian tanggung jawab antar class agar struktur data dan logika sistem terpisah secara jelas.

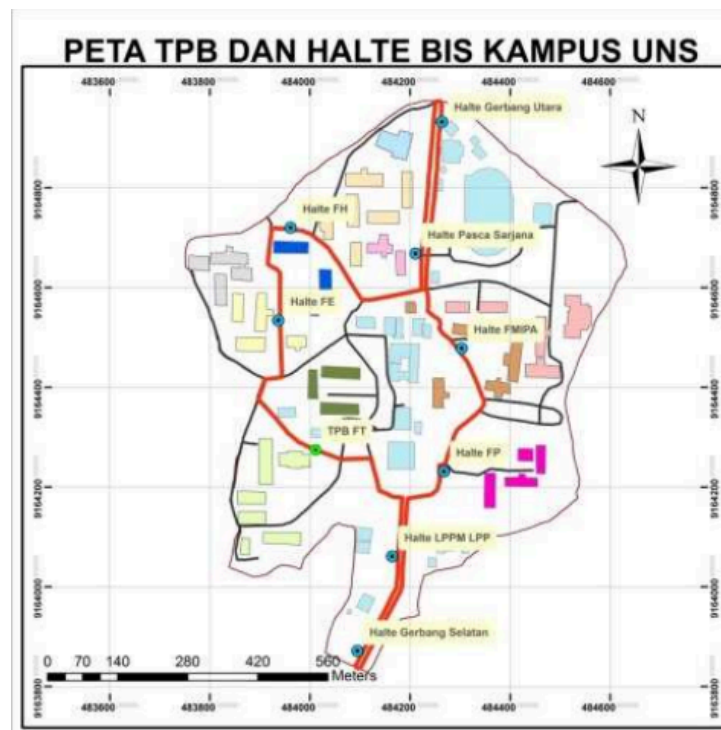
1. **Class Edge** : merepresentasikan sisi pada graph, menyimpan halte tujuan dan bobot waktu tempuh sebagai unit dasar penghubung antar halte.
2. **Class DisjoinSet** : mengelola pengelompokan halte ke dalam jalur barat atau timur. Constructor menetapkan setiap halte sebagai himpunan tunggal yang

berdiri sendiri, fungsi *find* menelusuri root suatu halte secara rekursif dengan teknik path compression, dan *union* menggabungkan dua halte ke dalam kelompok jalur yang sama.

3. **Class Project** : komponen utama yang menyimpan seluruh data sistem (graph, disjoint set, posisi dan status kapasitas bus) serta fungsi-fungsi pendukungnya. Fungsi *jalurA* memeriksa apakah suatu halte termasuk jalur Bus A menggunakan disjoint set, *tampilkanHalte* menampilkan daftar halte beserta status dan posisi tiap bus, sedangkan *hitungWaktu* bersama fungsi rekursif dfs menerapkan algoritma DFS untuk menghitung estimasi waktu tempuh antar halte. Inisialisasi graph dan disjoint set, beserta seluruh logika interaksi pengguna (peran supir dan penumpang), dijalankan langsung di dalam fungsi main.

5 Eksperimen & Analisis Kinerja (Minggu 14)

- Tracing:

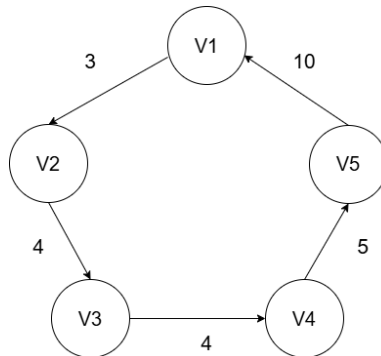


Source : <https://jurnal.uns.ac.id/matriks/article/view/37376/24601>

Gambar : Salah satu rekomendasi jalur bus UNS.

1. Tabel Adjacency (Daftar Sisi Graf)

- Data Halte Jalur Barat



Gambar : Graf sirkular berbobot berarah yang menunjukkan halte sebagai vertex dan menit (waktu tempuh) sebagai bobot pada jalur barat.

Keterangan :

V1 : Halte UNS Inn

V2 : Halte FT

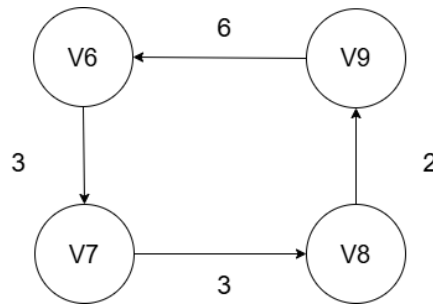
V3 : Halte FEB

V4 : Halte FH

V5 : Halte Pascasarjana

Edge	Bobot
Halte UNS Inn – Halte FT	3
Halte FT – Halte FEB	4
Halte FEB – Halte FH	4
Halte FH – Halte Pasca	5
Halte Pasca – Halte UNS Inn	10

- Data Halte Jalur Timur



Gambar : Graf sirkular berbobot berarah yang menunjukkan halte sebagai vertex dan menit (waktu tempuh) sebagai bobot pada jalur timur.

Keterangan :

V6 : Halte Gerbang Utara

V7 : Halte FK

V8 : Halte FMIPA

V9 : Halte FP

Edge	Bobot
Halte Gerbang Utara – Halte FK	3
Halte FK – Halte FMIPA	3
Halte FMIPA – Halte FP	2
Halte FP – Halte Gerbang Utara	6

2. Tracing Algoritma Depth-First Search (DFS) pada Skenario Penumpang dari Halte UNS Inn ke Halte FH

Iterasi	Halte saat ini	Halte Tujuan	Visited	Waktu Akumulasi	Keputusan
1	(1) UNS Inn	FH (4)	[UNS Inn]	0	cur ≠ tujuan, tandai UNS Inn visited, masuk ke tetangga FT

2	(2) FT	FH (4)	[UNS Inn, FT]	3	cur $\neq$ tujuan, tandai FT visited, masuk ke tetangga FEB
3	(3) FEB	FH (4)	[UNS Inn, FT, FEB]	7	cur $\neq$ tujuan, tandai FEB visited, masuk ke tetangga FH
4	(4) FH	FH (4)	[UNS Inn, FT, FEB, FH]	11	cur == tujuan!h[0], return

Contoh Deteksi Visited

Halte saat ini	Tetangga	Visited	Keputusan
Pasca (5)	UNS Inn (1)	✓	Skip, tidak ditelusuri

Karena UNS Inn sudah ditandai visited sebelumnya, DFS tidak menelusuri edge Pasca $\rightarrow$ UNS Inn untuk menghindari tak terhingga pada graf sirkular.

Hasil Tracing : Algoritma DFS menelusuri graf secara rekursif mulai dari halte asal hingga menemukan halte tujuan. Setiap halte yang dikunjungi langsung ditandai sebagai visited agar tidak dikunjungi ulang. Waktu tempuh terus diakumulasikan setiap berpindah halte sesuai bobot edge yang dilalui. Dengan cara ini, DFS dapat menemukan jalur dari UNS Inn ke FH dengan total waktu 11 menit tanpa terjebak dalam siklus graf sirkular.

3. Tracing Union Find - Disjoint Set

Proses Union Jalur Barat

Iterasi	Edge	Parent Sebelum Union	Keputusan
1	UNS Inn (1) - FT (2)	parent[2] = 2	union(1,2) $\rightarrow$ parent [2] = 1
2	FT (2) - FEB (3)	parent[3] = 3	union(2,3) $\rightarrow$ parent[3] = 1
3	FEB (3) - FH (4)	parent[4] = 4	union(3,4) $\rightarrow$ parent[4] = 1

Proses Union Jalur Timur

Iterasi	Edge	Parent Sebelum Union	Keputusan
1	Gerbang Utara (6) - FK (7)	parent[7] = 7	union(6,7) → parent [7] = 6
2	FK (7) - MIPA (8)	parent[8] = 8	union(7,8) → parent[8] = 6
3	MIPA (8) - FP (9)	parent[9] = 9	union(8,9) → parent[9] = 6
4	FP (9) - Gerbang Utara (6)	parent[6] = 6	find(9) = 6, find (6) = 6 → sama, skip

Contoh Deteksi Jalur Silang

Halte Asal	Halte Tujuan	find (asal)	find(tujuan)	Keputusan
FT (2)	FK (7)	1	6	find berbeda → jalur silang, ditolak!

Hasil Tracing:

Berdasarkan hasil tracing, proses union dilakukan mengikuti urutan rute bus sehingga seluruh halte dalam satu jalur memiliki root yang sama. Jalur barat memiliki root 1 (UNS Inn) dan jalur timur memiliki root 6 (Gerbang Utara). Ketika dua halte sudah memiliki root yang sama, operasi union di skip untuk menghindari redundansi. Struktur ini memungkinkan sistem mendeteksi jalur silang secara efisien, cukup dengan membandingkan nilai find() dari halte asal dan tujuan. Jika berbeda, sistem akan langsung menolak perjalanan tersebut.

- **Analisis Kompleksitas Intuitif:**

Estimasi Big-O sederhana (kasus terbaik/terburuk) berdasarkan loop atau struktur data yang digunakan.

1. DFS

Pada DFS kasus terbaik ketika pengguna memasukkan halte saat ini == halte tujuan dengan kompleksitas sebesar $O(1)$. Sedangkan kasus terburuk (worst case) adalah ketika pengguna menelusuri semua halte dan edge, sehingga kompleksitasnya sebesar $O(V+E)$.

2. Disjoint Set - Find

Kasus terbaiknya yaitu ketika node yang dicari sudah menjadi root (dirinya sendiri), maka kompleksitasnya menjadi $O(1)$. Namun, kasus terburuknya adalah ketika node yang dicari harus menelusuri satu persatu sebanyak V , maka kompleksitasnya menjadi $O(V)$.

3. Disjoint Set - Union

Operasi ini memanggil *find* dua kali sebelum melakukan penggabungan. Kasus terbaiknya yaitu ketika kedua halte sudah memiliki root yang sama atau root-nya mudah ditemukan, maka kompleksitasnya sebesar $O(1)$. Sedangkan kasus terburuknya mengikuti kasus terburuk pada *find*, yaitu ketika rantai parent yang ditelusuri cukup panjang dan kompleksitasnya menjadi $O(V)$.

4. Rute A

Saat pengguna memilih perjalanan melalui Jalur Barat (halte 1–5), program akan melakukan pengecekan jalur menggunakan Disjoint Set dan menghitung waktu perjalanan menggunakan DFS. Kasus terbaik terjadi ketika halte asal sama dengan posisi bus atau halte tujuan langsung ditemukan sehingga kompleksitasnya $O(1)$. Kasus terburuk terjadi ketika DFS harus menelusuri seluruh halte, sehingga kompleksitasnya $O(V+E)$.

5. Rute B

Saat pengguna memilih perjalanan melalui Jalur Timur (halte 6–9), akan terjadi pengecekan jalur dan perhitungan waktu menggunakan DFS. Kasus terbaik terjadi ketika tujuan langsung ditemukan atau halte asal sama dengan posisi bus sehingga kompleksitasnya $O(1)$. Kasus terburuk terjadi ketika seluruh halte pada Jalur Timur harus ditelusuri, sehingga kompleksitasnya $O(V+E)$.

6. Menampilkan halte

Fungsi `tampilkanHalte()` melakukan perulangan untuk memeriksa setiap halte dan menentukan apakah halte tersebut termasuk Jalur Barat atau Timur. Kasus terbaik dan terburuknya sama karena seluruh halte selalu ditampilkan. Kompleksitasnya $O(V)$.

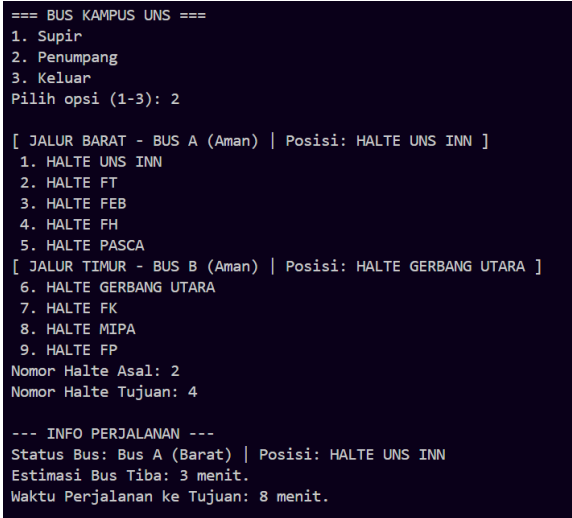
7. Inisialisasi graf

Pada proses inisialisasi graf, program membuat adjacency list untuk seluruh halte dan menambahkan edge sesuai rute Bus A dan Bus B. Kasus terbaik dan terburuknya sama karena seluruh halte dan rute selalu diproses satu kali saat program dijalankan. Sehingga, kompleksitasnya $O(V+E)$.

6 Pengujian & Validasi Solusi

- Skenario Uji

Berikut merupakan beberapa contoh skenario (kemungkinan) input dari pengguna

No	Gambar dan Keterangan
1.	Gambar:  <pre>=== BUS KAMPUS UNS === 1. Supir 2. Penumpang 3. Keluar Pilih opsi (1-3): 2 [JALUR BARAT - BUS A (Aman) Posisi: HALTE UNS INN] 1. HALTE UNS INN 2. HALTE FT 3. HALTE FEB 4. HALTE FH 5. HALTE PASCA [JALUR TIMUR - BUS B (Aman) Posisi: HALTE GERBANG UTARA] 6. HALTE GERBANG UTARA 7. HALTE FK 8. HALTE MIPA 9. HALTE FP Nomor Halte Asal: 2 Nomor Halte Tujuan: 4 --- INFO PERJALANAN --- Status Bus: Bus A (Barat) Posisi: HALTE UNS INN Estimasi Bus Tiba: 3 menit. Waktu Perjalanan ke Tujuan: 8 menit.</pre>

Keterangan:

Pengguna memilih menu sebagai Penumpang. Kemudian, program menampilkan daftar halte yang tersedia pada dua jalur. Selanjutnya, pengguna memilih Halte FT sebagai halte asal dan Halte FH sebagai halte tujuan. Input pengguna valid dan perjalanan dapat dilakukan dengan estimasi waktu tunggu hingga bus tiba di Halte FT adalah tiga menit dan durasi perjalanan dari Halte FT ke Halte FH yaitu delapan menit.

2. Gambar:

```

=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 2

[ JALUR BARAT - BUS A (Aman) | Posisi: HALTE UNS INN ]
1. HALTE UNS INN
2. HALTE FT
3. HALTE FEB
4. HALTE FH
5. HALTE PASCA
[ JALUR TIMUR - BUS B (Aman) | Posisi: HALTE GERBANG UTARA ]
6. HALTE GERBANG UTARA
7. HALTE FK
8. HALTE MIPA
9. HALTE FP
Nomor Halte Asal: 4
Nomor Halte Tujuan: 8
Rute Berbeda! Bus A (1-5) | Bus B (6-9)

```

Keterangan:

Pengguna memilih menu sebagai Penumpang. Kemudian, program akan menampilkan daftar halte yang tersedia pada dua jalur. Selanjutnya, pengguna memilih Halte FH sebagai halte asal dan Halte MIPA sebagai halte tujuan. Halte FH berada di Jalur Barat (Bus A) dan Halte MIPA berada di Jalur Timur (Bus B). Karena halte asal dan halte tujuan berada pada jalur yang berbeda, program menampilkan peringatan bahwa rute berbeda.

3. Gambar:

```

=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 2

[ JALUR BARAT - BUS A (Aman) | Posisi: HALTE UNS INN ]
1. HALTE UNS INN
2. HALTE FT
3. HALTE FEB
4. HALTE FH
5. HALTE PASCA
[ JALUR TIMUR - BUS B (Aman) | Posisi: HALTE GERBANG UTARA ]
6. HALTE GERBANG UTARA
7. HALTE FK
8. HALTE MIPA
9. HALTE FP
Nomor Halte Asal: 7
Nomor Halte Tujuan: 7
Halte asal dan tujuan sama.

```

Keterangan:

Pengguna memilih menu sebagai Penumpang. Kemudian, program akan menampilkan daftar halte yang tersedia pada dua jalur. Selanjutnya, pengguna memilih Halte FK sebagai halte asal dan Halte FK sebagai halte tujuan. Karena halte asal dan halte tujuan sama, program akan menampilkan peringatan.

4. Gambar:

```

=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 2

[ JALUR BARAT - BUS A (Aman) | Posisi: HALTE UNS INN ]
1. HALTE UNS INN
2. HALTE FT
3. HALTE FEB
4. HALTE FH
5. HALTE PASCA
[ JALUR TIMUR - BUS B (Aman) | Posisi: HALTE GERBANG UTARA ]
6. HALTE GERBANG UTARA
7. HALTE FK
8. HALTE MIPA
9. HALTE FP
Nomor Halte Asal: 10
Nomor Halte Tujuan: 2
Halte tidak valid.

```

Keterangan:

Pengguna memilih menu sebagai Penumpang. Kemudian, program akan menampilkan daftar halte yang tersedia pada dua jalur. Selanjutnya, pengguna memilih halte dengan indeks 10 sebagai halte asal dan Halte FT sebagai halte tujuan. Sistem hanya menyediakan sembilan halte. Karena pengguna memasukkan angka 10 pada kolom halte asal, sistem langsung mendeteksi input tersebut salah dan memunculkan pesan bahwa halte tidak valid.

5. Gambar:

```

=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 1
Armada (1: Bus A, 2: Bus B): 1
Kapasitas (1: Penuh, 0: Aman): 1

[ JALUR BARAT - BUS A (PENUH) | Posisi: HALTE UNS INN ]
1. HALTE UNS INN
2. HALTE FT
3. HALTE FEB
4. HALTE FH
5. HALTE PASCA
[ JALUR TIMUR - BUS B (Aman) | Posisi: HALTE GERBANG UTARA ]
6. HALTE GERBANG UTARA
7. HALTE FK
8. HALTE MIPA
9. HALTE FP
Masukkan Nomor Halte Sekarang: 3
Data berhasil diperbarui.

```

Keterangan:

Pengguna memilih menu sebagai Supir dengan armada Bus A dan mengubah kapasitas bus menjadi penuh. Sistem kemudian menampilkan daftar halte yang termasuk dalam Jalur Barat yang dilayani Bus A. Saat ditampilkan, posisi Bus A berada di Halte UNS Inn. Selanjutnya, supir memasukkan nomor halte 3 yang berarti Bus A saat ini berada di Halte FEB. Setelah input

diterima, sistem akan memperbarui data posisi Bus A dari Halte UNS Inn menjadi Halte FEB dan menyimpan status bus sebagai penuh. Lalu muncul pesan “Data Berhasil Diperbarui.”

6. Gambar:

```
=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 1
Armada (1: Bus A, 2: Bus B): 1
Kapasitas (1: Penuh, 0: Aman): 0

[ JALUR BARAT - BUS A (PENUH) | Posisi: HALTE FEB ]
1. HALTE UNS INN
2. HALTE FT
3. HALTE FEB
4. HALTE FH
5. HALTE PASCA
[ JALUR TIMUR - BUS B (Aman) | Posisi: HALTE GERBANG UTARA ]
6. HALTE GERBANG UTARA
7. HALTE FK
8. HALTE MIPA
9. HALTE FP
Masukkan Nomor Halte Sekarang: 4
Data berhasil diperbarui.
```

Keterangan:

Pengguna berperan sebagai Supir. Supir memilih Bus A dan mengubah status menjadi aman. Sebelumnya, Bus A berada di Halte FEB dengan status penuh. Setelah Supir memasukkan Halte FH (4), sistem memperbarui posisi Bus A ke Halte FH dan menyimpan status bus sebagai aman.

7. Gambar:

```
=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 2

[ JALUR BARAT - BUS A (PENUH) | Posisi: HALTE UNS INN ]
1. HALTE UNS INN
2. HALTE FT
3. HALTE FEB
4. HALTE FH
5. HALTE PASCA
[ JALUR TIMUR - BUS B (Aman) | Posisi: HALTE GERBANG UTARA ]
6. HALTE GERBANG UTARA
7. HALTE FK
8. HALTE MIPA
9. HALTE FP
Nomor Halte Asal: 1
Nomor Halte Tujuan: 4

--- INFO PERJALANAN ---
Status Bus: Bus A (Barat) | Posisi: HALTE UNS INN
Bus PENUH! Disarankan jalan kaki ke halte berikutnya
```

Keterangan:

Pengguna memilih peran sebagai Penumpang dan ingin melakukan perjalanan dari Halte UNS Inn ke Halte FH. Berdasarkan data yang tersimpan, Bus A sedang berada di Halte UNS Inn dan berstatus penuh. Sehingga, program menampilkan pesan agar penumpang disarankan berjalan ke halte berikutnya untuk menunggu bus (diasumsikan di halte berikutnya bus berstatus aman)

- **Kasus Batas:**

No	Gambar dan Keterangan
----	-----------------------

1. Gambar:

```
=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): a
Input tidak valid. Masukkan angka 1-3.

=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 5
Pilihan tidak valid. Masukkan angka 1-3.
```

Keterangan:

Ketika pengguna memasukkan inputan berupa huruf maka program akan menampilkan input tidak valid. Sedangkan jika pengguna memasukkan angka yang tidak tercantum (1, 2, 3), maka program akan mengeluarkan pernyataan “Pilihan tidak valid. Masukkan angka 1-3”

2. Gambar:

```
=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 1
Armada (1: Bus A, 2: Bus B): 4
Armada tidak valid. Pilih 1 atau 2.
```

Keterangan:

Jika pengguna memasukkan pilihan sebagai supir (1), lalu program akan

berjalan dan memberikan perintah kepada pengguna untuk memasukan armada yang dikendarai, jika armada yang dikendarai tidak sesuai yang tertulis (bukan 1 atau 2), maka program akan mengeluarkan pernyataan “Armada tidak valid. Pilih 1 atau 2”.

3. Gambar:

```
=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 1
Armada (1: Bus A, 2: Bus B): 1
Kapasitas (1: Penuh, 0: Aman): 5
Kapasitas tidak valid. Pilih 0 atau 1.
```

Keterangan:

Ketika pengguna yang berperan sebagai supir sudah memasukkan opsi dan armada dengan benar, maka program akan lanjut memberikan opsi kapasitas, jika pengisian kapasitas tidak sesuai, maka program akan menampilkan “Kapasitas tidak valid. Pilih 0 atau 1.”

4. Gambar:

```
=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 1
Armada (1: Bus A, 2: Bus B): 1
Kapasitas (1: Penuh, 0: Aman): 1

[ JALUR BARAT - BUS A (Aman) | Posisi: HALTE UNS INN ]
1. HALTE UNS INN
2. HALTE FT
3. HALTE FEB
4. HALTE FH
5. HALTE PASCA
[ JALUR TIMUR - BUS B (Aman) | Posisi: HALTE GERBANG UTARA ]
6. HALTE GERBANG UTARA
7. HALTE FK
8. HALTE MIPA
9. HALTE FP
Masukkan Nomor Halte Sekarang: 10
Halte tidak valid.
```

Keterangan:

Jika supir memasukkan halte saat ini yang disinggahi namun halte tidak tercatat, maka program akan menampilkan “Halte tidak valid.”

5. Gambar:

```

=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 2

[ JALUR BARAT - BUS A (Aman) | Posisi: HALTE UNS INN ]
1. HALTE UNS INN
2. HALTE FT
3. HALTE FEB
4. HALTE FH
5. HALTE PASCA
[ JALUR TIMUR - BUS B (Aman) | Posisi: HALTE GERBANG UTARA ]
6. HALTE GERBANG UTARA
7. HALTE FK
8. HALTE MIPA
9. HALTE FP
Nomor Halte Asal: 1
Nomor Halte Tujuan: 10
Halte tidak valid.

```

Keterangan:

Jika pengguna berperan sebagai penumpang dan halte asal dan atau halte tujuan tidak sesuai dengan pilihan (kurang dari 1 atau lebih dari 9), maka program akan menampilkan “Halte tidak valid.”

6. Gambar:

```

=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3): 2

[ JALUR BARAT - BUS A (Aman) | Posisi: HALTE UNS INN ]
1. HALTE UNS INN
2. HALTE FT
3. HALTE FEB
4. HALTE FH
5. HALTE PASCA
[ JALUR TIMUR - BUS B (Aman) | Posisi: HALTE GERBANG UTARA ]
6. HALTE GERBANG UTARA
7. HALTE FK
8. HALTE MIPA
9. HALTE FP
Nomor Halte Asal: a
Input tidak valid. Masukkan angka 1-9.

```

Keterangan:

Ketika penumpang memasukkan halte asal naik tetapi mengisi nomor halte asal menggunakan huruf (string), maka program akan menampilkan “Input tidak valid. Masukkan angka 1-9.”

7 Kesimpulan & Refleksi

- Jawaban terhadap rumusan masalah.

Sistem yang dikembangkan berhasil menjawab permasalahan. Operasional dua jalur bus (barat dan timur) yang sebelumnya tidak memiliki sistem informasi, kini dapat dilihat melalui Graph, di mana setiap halte menjadi vertex dan hubungan antar halte menjadi edge yang berbobot waktu tempuh dalam satuan menit. Algoritma DFS diterapkan pada fungsi `hitungWaktu()`, sistem dapat menghitung estimasi waktu kedatangan bus dan durasi perjalanan antar halte, sehingga penumpang tidak mengandalkan perkiraan manual yang bisa jadi salah. Penggunaan Union-Find (Disjoint Set) juga menangani validasi rute, karena sistem otomatis menolak perjalanan lintas jalur. Selain itu, informasi status bus (penuh/aman) yang ditampilkan dapat memberikan informasi kepada penumpang tentang kondisi armada sebelum memutuskan untuk menunggu atau berjalan ke halte berikutnya.

- Refleksi mengenai peran struktur data dalam efektivitas penyelesaian proyek.

Pemilihan struktur data pada proyek ini terbukti dapat menyelesaikan masalah dengan efektif. Graph dipilih karena dapat merepresentasikan hubungan antar halte, sehingga penelusuran rute dan perhitungan waktu menjadi lebih sistematis. ArrayList digunakan karena setiap halte menyimpan daftar Edge berupa halte tujuan dan bobot waktu tempuhnya, mengingat jumlah edge yang kecil dan tetap. DFS terbukti efektif karena rute bersifat sirkular dan tidak bercabang, sehingga setiap penelusuran hanya menyusuri satu jalur tunggal hingga tujuan ditemukan. Union-Find dapat menghasilkan solusi yang efisien untuk masalah pengelompokan halte ke dalam dua jalur terpisah.

Kontribusi Anggota

Nama Anggota	Peran dan Kontribusi
Grecia Aulia Hermawan	Pencarian informasi bus UNS, perancangan solusi, pembuatan kode, pembuatan laporan, pembuatan ppt.
Stiven Arya Duta Wicaksono	Analisis kompleksitas, pembuatan kode, pembuatan laporan, pembuatan ppt.
Naila Zhafira Rahma Widyatmoko	Tracing, uji kode, pembuatan kode, pembuatan laporan, pembuatan ppt.

Daftar Pustaka

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.

Tamimi, D. K., Handayani, D., & MHM, A. (2014). PERENCANAAN TEMPAT PERHENTIAN BIS KAMPUS DALAM RANGKA Mendukung PROGRAM GREEN CAMPUS UNS. *Matriks Teknik Sipil*, 2. <https://doi.org/10.20961/mateksi.v2i4.37376>

Weiss, M. A. (2012). *Data Structures and Algorithm Analysis in Java*. Pearson.

Lampiran

- Cuplikan kode program utama. Berikut merupakan beberapa tangkapan layar program utama

```
static class DisjointSet {
    int[] parent;
    DisjointSet(int n) {
        parent = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }
    int find(int x) {
        if (parent[x] == x) {
            return x;
        }
        return parent[x] = find(parent[x]);
    }
    void union(int x, int y) {
        int px = find(x);
        int py = find(y);
        if (px != py) {
            parent[py] = px;
        }
    }
}
```

```

static ArrayList<Edge>[] graph = new ArrayList[TOTAL];
static DisjointSet ds = new DisjointSet(TOTAL);

static boolean jalurA(int a) {
    if (ds.find(a) == ds.find(x: 1)) {
        return true;
    } else {
        return false;
    }
}

static int hitungWaktu(int dari, int ke) {
    if (dari == ke) {
        return 0;
    }
    int[] h = {-1};
    dfs(dari, ke, w: 0, new boolean[TOTAL], h);
    return h[0];
}

```

```

static void dfs(int cur, int tujuan, int w, boolean[] v, int[] h) {
    if (cur == tujuan) {
        h[0] = w;
        return;
    }
    v[cur] = true;
    for (Edge e : graph[cur]) {
        if (!v[e.tujuan]) {
            dfs(e.tujuan, tujuan, w + e.waktu, v, h);
            if (h[0] != -1) {
                return;
            }
        }
    }
}

```

- Hasil tangkapan layar (screenshot) *running program*.

```

=== BUS KAMPUS UNS ===
1. Supir
2. Penumpang
3. Keluar
Pilih opsi (1-3):

```